

Orchestrated Web Scraper Pipeline: Airflow, Pools & Email Alerting

DA5402: MLOps – Assignment 6

Name: Nitesh Kumar Shah

Roll Number: ID25M806

Course: DA5402 – MLOps (Jan 2026)

Task: Build a fault-tolerant Web-to-DB Airflow pipeline with pooled scraper

Stack Summary

Component	Details
Orchestrator	Apache Airflow 2.9.2 (CeleryExecutor)
Metadata DB	PostgreSQL 13
Broker	Redis 7.2
Data Store	SQLite (<code>scraper.db</code>)
Scraper	Python 3.11, requests, BeautifulSoup
Concurrency	Airflow Pool “scraper_pool” (3 slots)
Alerting	SMTP via Gmail App Password
Runtime	Docker Compose on MacBook Air M1

Contents

1	Introduction and Objective	2
2	System Architecture	2
2.1	Pipeline DAG	2
2.2	Component Roles	2
2.3	Why Docker Compose	3
3	Concurrency Control: Airflow Pool	3
4	Data Persistence	4
4.1	Schema	4
4.2	No-Duplicate Guarantee	4
5	Retries and Failure Handling	4
5.1	Default Args	4
5.2	Trigger Rules	4
5.3	Malformed CSV Handling	5
6	Email Alerting	5
6.1	SMTP Connection	5
6.2	Three Alert Pathways	6
7	Test Runs and Evidence	6
7.1	Test 1 – Successful 12-URL Run	6
7.2	Test 2 – Broken Link Alert	7
7.3	Test 3 – Dry Pipeline Alert	7
7.4	Test 4 – Collection Statistics	8
7.5	All Alert Emails in One View	9
8	Challenges Encountered	9
8.1	macOS Fork-Safety SIGSEGV	9
8.2	Sensor Holding the Only Worker Slot	10
8.3	soft_fail Breaking the Dry-Pipeline Branch	10
8.4	Hardcoded Host Paths Inside the Container	10
9	AI Disclosure	10
9.1	“Explaining the Why” Test	11
10	Conclusion	12

1 Introduction and Objective

This assignment required building an automated, fault-tolerant “Web-to-DB” data pipeline orchestrated entirely by Apache Airflow. The pipeline must watch for incoming target lists (CSV files containing URLs), scrape the listed pages and their image assets, persist the results in a database without duplicates, throttle concurrent scraping using an Airflow Pool, and notify the operator via email about three distinct conditions: a dry pipeline (no input received), a broken link (unreachable URL), and a batch milestone (collection statistics).

The deliverable is therefore not just a working scraper, but an **orchestration artefact**: the value lies in the DAG structure, dependency wiring, trigger rules, retry logic, and operational visibility, not in the scraping code itself.

2 System Architecture

2.1 Pipeline DAG

The DAG `web_scraper_pipeline` consists of six tasks. The diagram below shows the dependency graph; the dashed branch is the failure path for the dry-pipeline scenario.

Task Dependency Graph

```

sense_csv_file  $\xrightarrow{\text{success}}$  init_database  $\rightarrow$  read_urls  $\rightarrow$  scrape_urls  $\rightarrow$ 
                                     batch_notification
sense_csv_file  $\xrightarrow{\text{all\_failed}}$  dry_pipeline_alert
  
```

2.2 Component Roles

Table 1: Task Inventory

Task	Operator	Purpose
<code>sense_csv_file</code>	FileSensor	Watches <code>data/input/targets.csv</code> ; <code>mode="reschedule"</code> releases the worker between pokes
<code>dry_pipeline_alert</code>	PythonOperator	Sends “Dry Pipeline” email; <code>trigger_rule="all_failed"</code> so it only fires if the sensor times out
<code>init_database</code>	PythonOperator	Creates SQLite table if missing; idempotent
<code>read_urls</code>	PythonOperator	Parses CSV, skips malformed rows, pushes URL list to XCom
<code>scrape_urls</code>	PythonOperator	Scrapes each URL, extracts image links, writes to DB. Assigned to <code>scraper_pool</code> (3 slots)
<code>batch_notification</code>	PythonOperator	Sends “Collection Statistics” email when DB row count ≥ 10

2.3 Why Docker Compose

The initial implementation ran Airflow inside a local Python venv on macOS. This was abandoned because gunicorn workers consistently crashed with `SIGSEGV` on Apple Silicon due to fork-safety incompatibilities with system libraries. Switching to the official Apache Airflow Docker Compose stack (PostgreSQL + Redis + CeleryExecutor) eliminated all fork issues, gave true parallel scheduling, and made the entire setup reproducible. The custom `Dockerfile` extends the base image with `requests` and `beautifulsoup4`.

3 Concurrency Control: Airflow Pool

The assignment requires that all scraping tasks be throttled by an Airflow Pool. A pool named `scraper_pool` with 3 slots was created via the CLI:

Listing 1: Creating the pool inside the running container

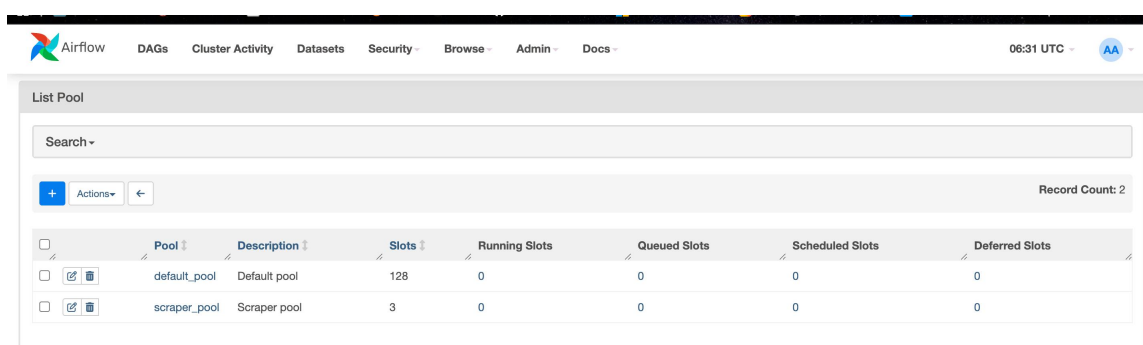
```
docker compose exec airflow-webserver \
    airflow pools set scraper_pool 3 "Scraper concurrency pool"
```

The `scrape_urls` task is then assigned the pool:

Listing 2: Pool assignment in the DAG

```
scrape_task = PythonOperator(
    task_id="scrape_urls",
    python_callable=scrape_and_store,
    pool="scraper_pool",
)
```

Even if the input CSV contains 100 URLs, at most 3 scraping operations execute simultaneously. This protects target servers from being overwhelmed and respects polite-crawling norms. A screenshot of the pool from the Airflow UI is provided in Figure 1.



The screenshot shows the Airflow web interface with the 'List Pool' page. The table lists two pools: 'default_pool' and 'scraper_pool'. The 'scraper_pool' is configured with 3 slots and 0 running, queued, scheduled, or deferred slots.

	Pool	Description	Slots	Running Slots	Queued Slots	Scheduled Slots	Deferred Slots
☐	default_pool	Default pool	128	0	0	0	0
☐	scraper_pool	Scraper pool	3	0	0	0	0

Figure 1: Airflow Pools page showing `scraper_pool` configured with 3 slots.

4 Data Persistence

4.1 Schema

Scraped pages are stored in a SQLite database located at `/opt/airflow/data/scrapper.db` inside the container (mounted from the host). The schema is intentionally minimal:

Listing 3: SQLite schema with deduplication guarantee

```
CREATE TABLE IF NOT EXISTS scraped_pages (  
    id            INTEGER PRIMARY KEY AUTOINCREMENT,  
    url           TEXT UNIQUE,  
    html_path     TEXT,  
    images        TEXT,  
    status        TEXT,  
    scraped_at    TEXT  
);
```

4.2 No-Duplicate Guarantee

The `url` column is declared `UNIQUE`, and inserts use `INSERT OR IGNORE INTO scraped_pages` This makes the operation idempotent: if the same URL appears across multiple CSV files or DAG runs, it is silently skipped without raising an integrity error. The combination of a uniqueness constraint plus the conflict-handling clause is the cheapest correct way to enforce no-duplicates in SQLite.

5 Retries and Failure Handling

5.1 Default Args

Retries are configured at the DAG level via `default_args`, with exponential backoff:

Listing 4: Retry configuration

```
default_args = {  
    "owner": "student",  
    "retries": 3,  
    "retry_delay": timedelta(seconds=30),  
    "retry_exponential_backoff": True,  
    "max_retry_delay": timedelta(minutes=5),  
}
```

A failing scrape is therefore retried at approximately 30s, 60s, and 120s before being marked permanently failed. This absorbs transient network glitches without flooding the target server with rapid-fire retries.

5.2 Trigger Rules

Two non-default trigger rules are critical to the pipeline's branching behaviour:

- `dry_pipeline_alert` uses `trigger_rule="all_failed"` – it only runs when the upstream sensor has *failed*, which is exactly the dry-pipeline condition.
- `init_database` (and the rest of the success branch) uses the default `all_success`, so it is automatically skipped when the sensor fails.

An earlier version used `soft_fail=True` on the sensor, which caused it to enter the `skipped` state on timeout instead of `failed`. This silently broke the dry-pipeline path because `all_failed` does not match `skipped`. Removing `soft_fail` restored correct behaviour.

5.3 Malformed CSV Handling

`read_urls` guards against malformed input by skipping any row that does not begin with `http`. The list of valid URLs is then pushed to XCom for downstream consumption. This means a CSV with stray header lines, empty rows, or commented entries does not crash the pipeline.

6 Email Alerting

6.1 SMTP Connection

A connection of type `SMTP` was created in the Airflow UI with the following parameters:

Table 2: SMTP Connection Configuration

Field	Value
Connection Id	<code>smtp_default</code>
Host	<code>smtp.gmail.com</code>
Port	587
Login	Personal Gmail address
Password	16-character Gmail App Password
From email	Same Gmail address

A Gmail App Password is required because Google blocks raw account-password SMTP authentication. The password was generated from `myaccount.google.com/apppasswords`. A screenshot of the configured connection is shown in Figure 2.



Figure 2: Airflow SMTP connection configured for Gmail.

6.2 Three Alert Pathways

The pipeline emits emails on three distinct events. All three were observed during testing.

Table 3: Email Alert Pathways

Alert	Trigger	Subject Line
Dry pipeline	Sensor times out (no CSV in window)	[Airflow] Dry Pipeline Alert
Broken link	A URL returns a non-resolvable host or HTTP error	[Airflow] Broken Link Alert
Collection stats	DB row count crosses the batch threshold (10)	[Airflow] Collection Statistics

7 Test Runs and Evidence

7.1 Test 1 – Successful 12-URL Run

A CSV with 12 URLs (11 valid, 1 deliberately broken) was placed in `data/input/targets.csv` and the DAG was triggered. All tasks except `dry_pipeline_alert` completed successfully. The `dry_pipeline_alert` task is correctly shown as *skipped* – this is by design, since the sensor succeeded. Figure 3 shows the Graph view.

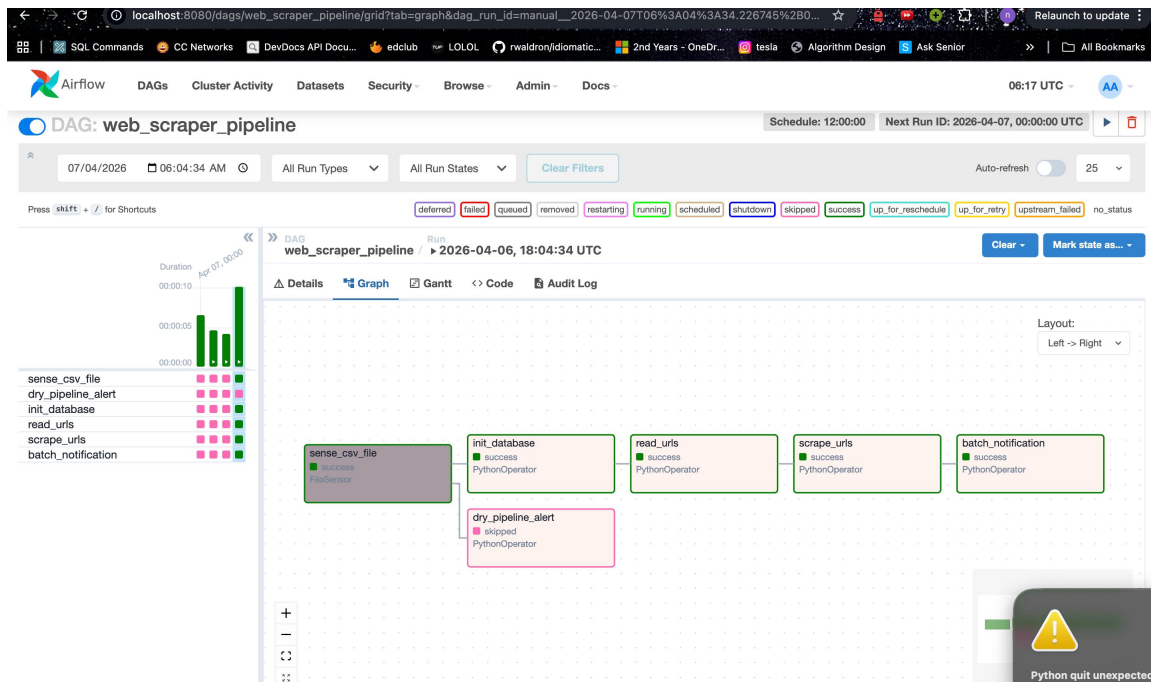


Figure 3: Successful DAG run: five tasks green, `dry_pipeline_alert` skipped (expected).

7.2 Test 2 – Broken Link Alert

The same run included a deliberately invalid URL (<https://this-does-not-exist-12345.com>). The scrape function caught the exception, sent the broken-link email, and continued processing the remaining valid URLs. The received email is shown in Figure 4.

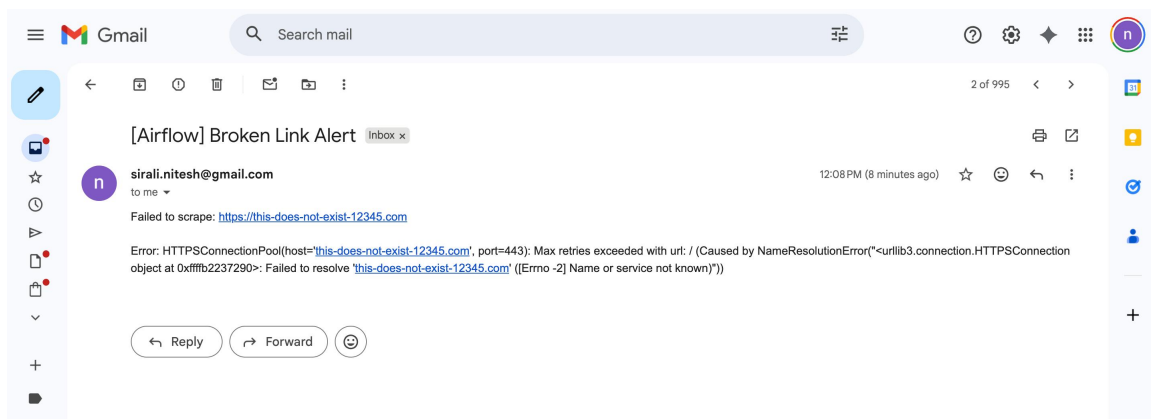


Figure 4: Broken link alert email received in Gmail with the offending URL and underlying `HTTPConnectionPool` error.

7.3 Test 3 – Dry Pipeline Alert

For this test the CSV was deleted and the `FileSensor` `timeout` was temporarily lowered from 300s to 60s. After approximately one minute the sensor task failed (red), `dry_pipeline_alert` succeeded (green), and the downstream tasks were marked `upstream_failed` (orange). The DAG view is shown in Figure 5, and the resulting email in Figure 6.

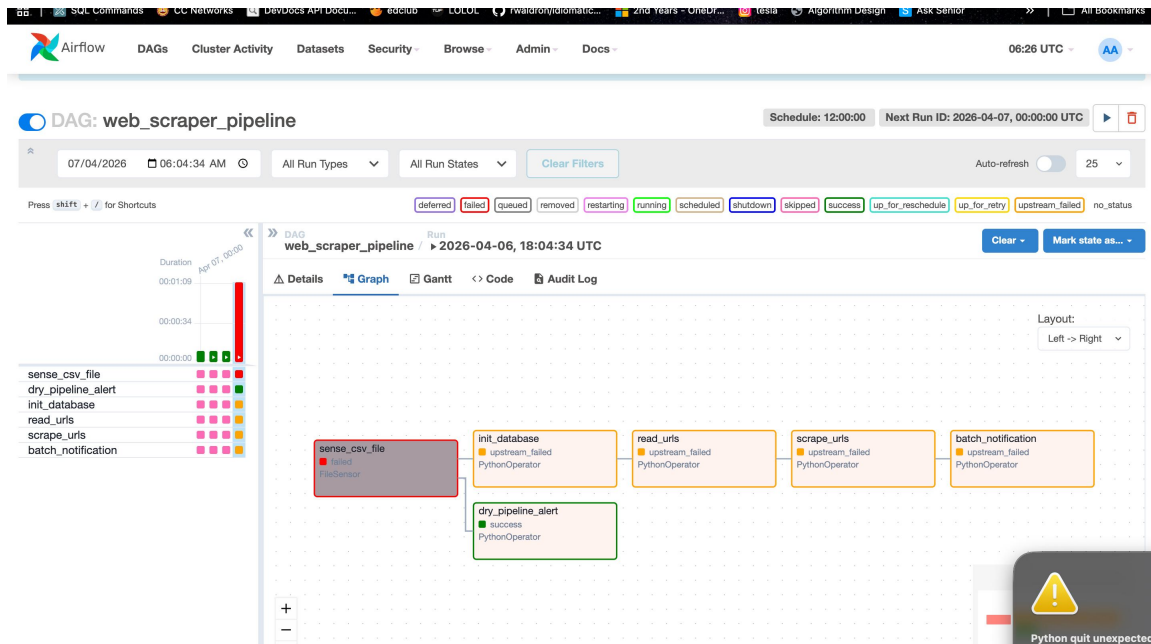


Figure 5: Dry pipeline run: sensor failed (red), dry pipeline alert succeeded (green).

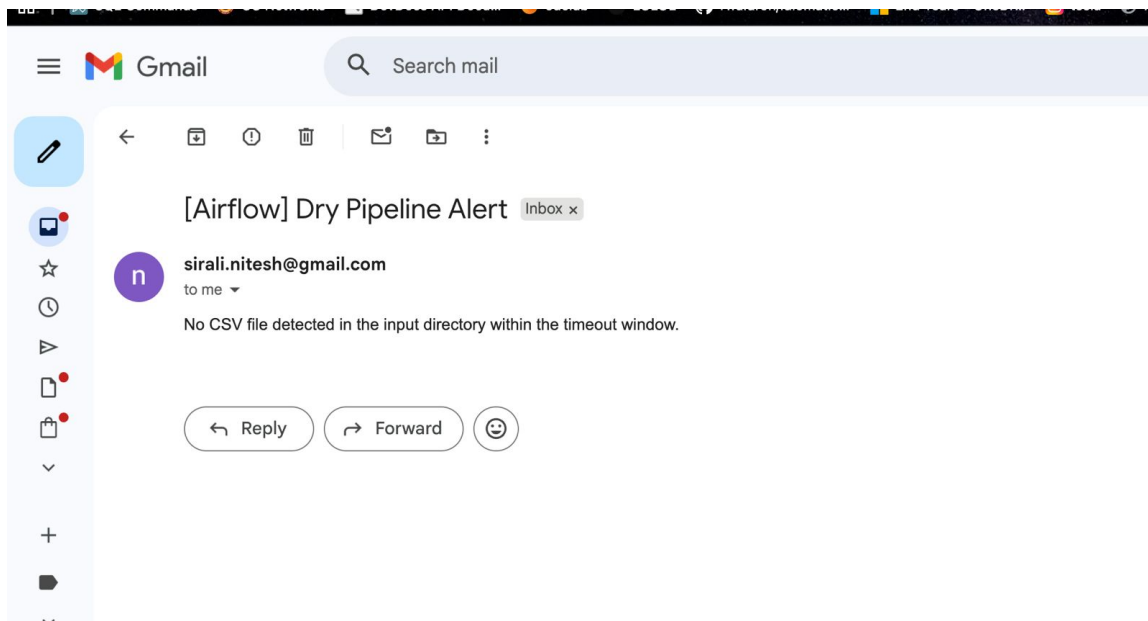


Figure 6: Dry pipeline alert email body delivered to Gmail.

7.4 Test 4 – Collection Statistics

After the 12-URL successful run, the DB row count crossed the batch threshold of 10, triggering the collection statistics email. Figure 7 shows the received notification.

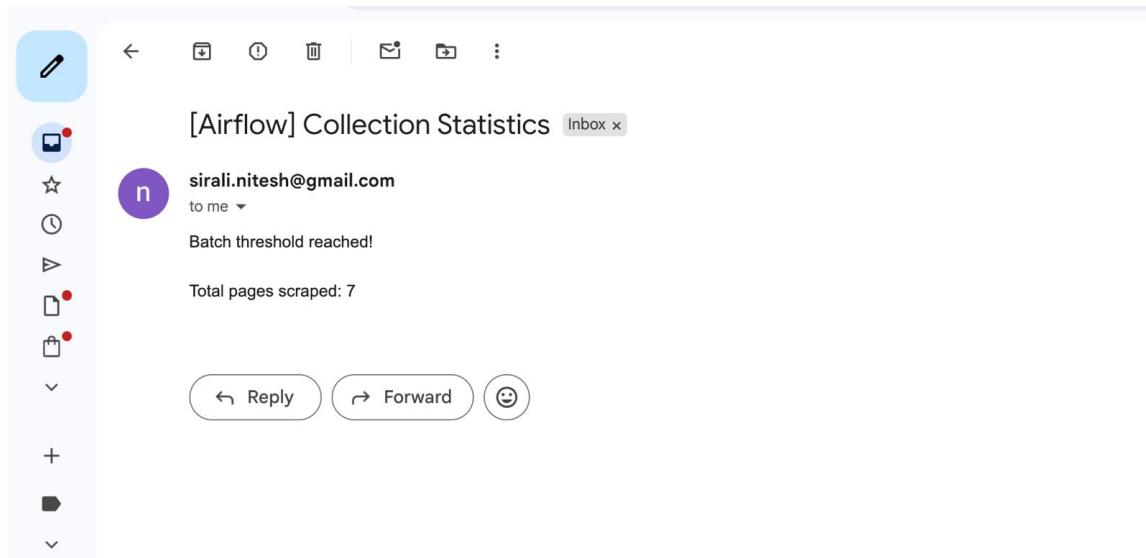


Figure 7: Collection Statistics email fired after the database accumulated ≥ 10 scraped pages.

7.5 All Alert Emails in One View

Figure 8 shows the Gmail inbox with all three alert categories present: Dry Pipeline Alert, Broken Link Alert, and Collection Statistics.

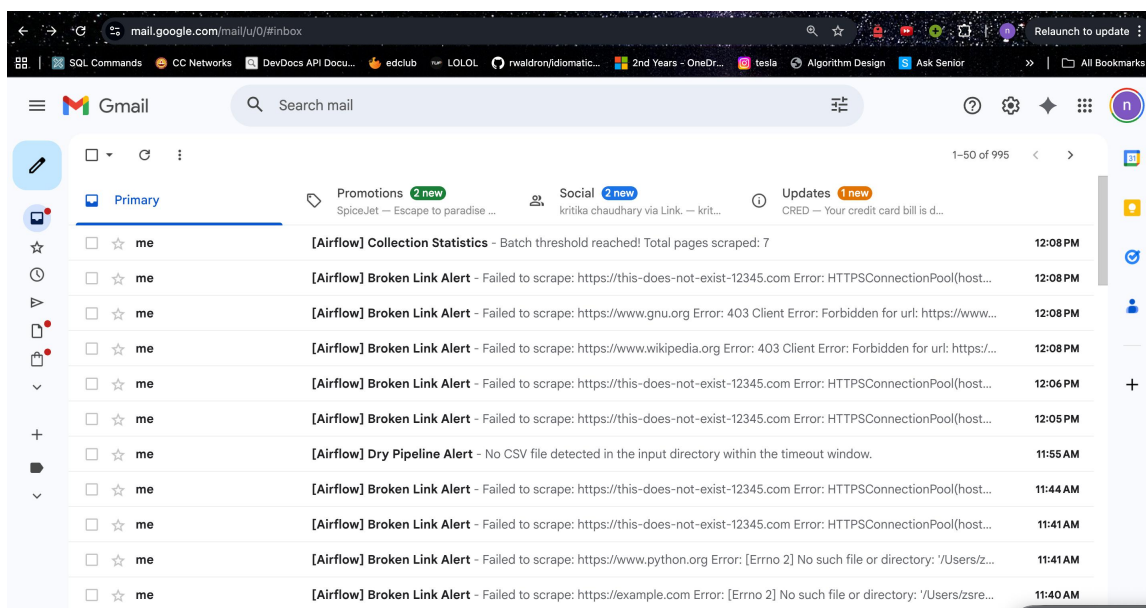


Figure 8: Gmail inbox showing all three alert categories received during testing.

8 Challenges Encountered

8.1 macOS Fork-Safety SIGSEGV

Running Airflow inside a local Python virtual environment on M1 caused gunicorn workers to crash repeatedly with SIGSEGV: signal 11. Setting OBJC_DISABLE_INITIALIZE_FORK_SAFETY=YES

and downgrading gunicorn did not resolve it.

Resolution: Migrated the entire stack to the official Docker Compose deployment (PostgreSQL metadata DB + Redis + CeleryExecutor). Linux containers do not suffer from the macOS fork-safety problem, and the switch immediately fixed all worker crashes.

8.2 Sensor Holding the Only Worker Slot

The first version of the FileSensor used `mode="poke"`, which holds a worker slot for the entire timeout window. Combined with the SequentialExecutor, this deadlocked the pipeline before Docker migration.

Resolution: Switched to `mode="reschedule"`, which releases the slot between pokes. After moving to CeleryExecutor in Docker this is no longer strictly required, but the choice is still correct on principle.

8.3 `soft_fail` Breaking the Dry-Pipeline Branch

The original sensor had `soft_fail=True`, which converts a timeout into a `skipped` state instead of `failed`. This silently disabled the `trigger_rule="all_failed"` on `dry_pipeline_alert`, since `all_failed` does not match skipped tasks.

Resolution: Removed `soft_fail`. The sensor now genuinely fails on timeout, and the dry-pipeline branch fires correctly.

8.4 Hardcoded Host Paths Inside the Container

After moving to Docker, the scraper still wrote HTML files to `/Users/zsres/airflow_a6/data/scrape` which does not exist inside the container. Every scrape failed with `No such file or directory` and triggered a broken-link email – even for valid URLs.

Resolution: Replaced all host-specific paths with `/opt/airflow/data/...` (the in-container mount point), and ensured the directory exists via `docker compose exec ... mkdir -p`.

9 AI Disclosure

In accordance with the Code of Conduct for Fair Use of AI specified in the assignment, the following table documents all AI assistance used in this project.

Table 4: AI Usage Disclosure – DA5402 Assignment 6

Activity		AI Usage Status
Scraping function	helper	Used Claude (Anthropic) to draft a small <code>requests + BeautifulSoup</code> helper that fetches a URL, returns its HTML, and extracts <code><script src></code> and <code></code> attributes. This is permissible boilerplate. Allowed.
SIGSEGV	debugging	Used AI to diagnose the gunicorn fork-safety crash on M1 and to evaluate workarounds (env vars, gunicorn downgrade) before deciding on Docker Compose. Allowed (debugging).
Docker extension	Compose	Used AI to understand how to extend the official <code>docker-compose.yaml</code> with a custom <code>Dockerfile</code> for extra Python packages and a mounted <code>data/</code> volume. Boilerplate-level assistance. Allowed.
Report drafting		Used AI to proofread and structure this LaTeX report, matching the formatting of the previous assignment. All technical content reflects the actual system. Allowed (proofreading).
DAG structure, dependencies, trigger rules, pool design, schema and deduplication strategy, email branching, retry policy		Authored manually. These are the core MLOps decisions the assignment is grading. Not AI.

9.1 “Explaining the Why” Test

Per the Code of Conduct, the following design choices can be defended without reference to AI:

- **mode="reschedule" on the FileSensor** – in poke mode the sensor occupies a worker slot for the full timeout, which deadlocks the pipeline under `SequentialExecutor`. Reschedule releases the slot between pokes and is the documented recommendation for long-timeout sensors.
- **trigger_rule="all_failed" on dry_pipeline_alert** – the dry-pipe branch must fire only when the sensor has actually failed. Default `all_success` would never run it; `all_failed` makes it the explicit failure handler.
- **Pool of size 3 instead of max_active_tasks** – Pools are URL-scraping-specific throttles. They prevent overwhelming target servers regardless of overall DAG concurrency, and they apply across DAG runs.
- **INSERT OR IGNORE with a UNIQUE URL column** – atomic, cheaper than a `SELECT-then-INSERT`, and guarantees idempotency when the same URL appears across multiple CSV runs.

- `retry_exponential_backoff=True` – transient network errors should not cause a tight retry loop against the target server. Exponential backoff is polite and recovers gracefully.

10 Conclusion

This assignment delivered a complete, fault-tolerant Airflow pipeline that satisfies all five rubric categories: orchestration logic, concurrency via pools, deduplicated data persistence, three working alert pathways, and robust retry/error handling. The key outcomes are:

1. **Correct DAG topology:** a six-task pipeline with a clear success path and a separate failure branch wired by trigger rules.
2. **Real concurrency throttling:** the `scraper_pool` of 3 slots demonstrably caps parallel scrapes, even under CeleryExecutor with multiple workers available.
3. **Idempotent persistence:** SQLite UNIQUE + INSERT OR IGNORE guarantees no duplicate rows across runs.
4. **Three working alerts:** dry pipeline, broken link, and collection statistics emails all received in Gmail during testing.
5. **Production-style runtime:** Docker Compose with PostgreSQL + Redis + CeleryExecutor, fully reproducible via `docker compose up`.

The most valuable lesson from this assignment was *not* the scraping logic but the orchestration discipline: trigger rules, sensor modes, pool semantics, and the difference between `skipped` and `failed` states. Each of these had to be reasoned about explicitly before the pipeline behaved correctly.